

Rapport de synthèse du projet

Planificateur de tâches avec alarme vocale

Projet réalisé avec Python, Django, SQLite, HTML, JavaScript, pyttsx3 et synthèse vocale française.

1. Présentation générale

Le présent projet consiste en la réalisation d'une application web de planification de tâches capable de déclencher automatiquement une alarme vocale lorsqu'une tâche arrive à échéance. L'utilisateur utilise une page web pour enregistrer le nom de la tâche, la date et l'heure prévues. Le système surveille ensuite la base de données et lit à haute voix le contenu de la tâche au moment programmé.

L'objectif principal est de fournir un outil simple, utile et fiable pour les rappels quotidiens : rendez-vous, activités professionnelles, prises de médicaments, réunions, travaux à rendre ou toute autre action qui nécessite une notification claire. La particularité du projet est que le rappel ne se limite pas à un affichage visuel : il est prononcé par le système afin d'être audible même lorsque l'utilisateur ne regarde pas l'écran.

L'application a été développée avec Django pour la partie web, SQLite pour le stockage des données, HTML et JavaScript pour l'interface utilisateur, ainsi que plusieurs technologies de synthèse vocale pour rendre les alarmes audibles et naturelles. La solution prend en charge plusieurs moteurs vocaux afin de rester fonctionnelle sur Windows, Linux et macOS.

2. Objectifs du projet

- Permettre à l'utilisateur de créer, consulter et supprimer des tâches planifiées depuis une interface web.
- Déclencher automatiquement chaque tâche arrivée à échéance, sans intervention manuelle.
- Lire clairement le nom de chaque tâche à haute voix en français.
- Garantir que plusieurs tâches prévues à la même minute soient lues les unes après les autres.
- Assurer une compatibilité multiplateforme avec des fallbacks vocaux robustes.
- Proposer une configuration vocale permettant d'ajuster la voix, la vitesse, le volume, la tonalité et la répétition.

3. Architecture technique

L'architecture du projet est organisée autour d'une application Django nommée task. Cette application contient les modèles de données, les formulaires, les vues, les routes, les templates HTML, le moteur de surveillance des alarmes et la logique de synthèse vocale. La base SQLite conserve les tâches et la configuration vocale globale.

La page web sert uniquement à planifier les tâches et à afficher leur état. Le déclenchement réel de l'alarme est effectué côté backend, ce qui rend le système plus fiable : même si l'utilisateur ne clique sur aucun bouton, la boucle de surveillance vérifie régulièrement la base de données et envoie les tâches dues au thread vocal.

3.1 Modèles de données

Le modèle Task représente une tâche planifiée. Il contient le nom de la tâche, la date, l'heure, l'état actif ou terminé et la date de création. Une propriété `due_datetime_local` combine la date et l'heure pour produire une échéance cohérente avec le fuseau horaire utilisé par Django.

Le modèle VoiceConfig représente une configuration vocale unique. Il centralise les paramètres tels que le moteur vocal, la voix française, la voix neurale premium, le débit, le volume, la tonalité, le gain audio, le nombre de répétitions, le texte de pré-alarme et l'activation du fallback système. Une méthode `get_solo` garantit qu'il n'existe qu'une seule configuration active.

3.2 Flux de fonctionnement

- L'utilisateur crée une tâche depuis le formulaire web.
- Django valide les données et enregistre la tâche dans SQLite.
- Le scheduler backend vérifie chaque seconde les tâches encore actives.
- Lorsqu'une échéance est atteinte, la tâche est placée dans une file vocale thread-safe.
- Un thread dédié à l'audio lit les alarmes dans l'ordre d'arrivée.
- Après une lecture réussie, la tâche est désactivée pour éviter les répétitions infinies.

3.3 Choix de Django et SQLite

Django a été choisi parce qu'il fournit une structure claire pour gérer les routes, les vues, les formulaires et l'administration. Son ORM facilite l'accès à la base de données sans écrire de requêtes SQL manuelles. SQLite convient bien à ce type de projet, car il est léger, intégré, facile à déployer et suffisant pour une application locale ou de petite taille.

3.4 Interface utilisateur

L'interface HTML permet de créer rapidement une alarme, de visualiser les tâches programmées, de suivre un compte à rebours et de supprimer une tâche. JavaScript met à jour l'horloge et le décompte côté

navigateur afin de rendre l'interface vivante. La lecture automatique principale reste néanmoins gérée par Python afin d'être indépendante du navigateur.

4. Système d'alarme vocale

La partie la plus importante du projet est le système vocal. Le besoin était de garantir que chaque tâche arrivée à échéance soit réellement lue à haute voix, avec une voix française audible. Pour cela, le projet utilise une stratégie en plusieurs niveaux : une voix neurale premium lorsque les dépendances et Internet sont disponibles, puis pyttsx3 pour la synthèse hors ligne, puis les moteurs système comme espeak-ng, spd-say, PowerShell SAPI ou say selon la plateforme.

4.1 Thread vocal non bloquant

La synthèse vocale peut être lente ou bloquante. Pour éviter de figer l'application web, les alarmes sont placées dans une file d'attente et traitées par un thread séparé. Cette organisation permet à Django de continuer à répondre aux requêtes pendant que la voix parle. La file garantit aussi que plusieurs tâches prévues au même instant ne se mélangent pas : elles sont lues l'une après l'autre.

4.2 Sélection de la voix française

Le module voice.py inspecte les voix disponibles sur la machine et privilégie les voix françaises. Sur Windows, le système peut utiliser SAPI5 et les voix installées dans les paramètres de langue. Sur Linux, pyttsx3 s'appuie généralement sur espeak. Sur macOS, le driver natif nssm ou la commande say peuvent être utilisés. Lorsque la voix française locale n'est pas détectée, l'application affiche un message explicite et tente un fallback système.

4.3 Amélioration de la qualité audio

Pour obtenir une voix plus agréable, le projet intègre edge-tts, qui permet d'utiliser des voix neurales françaises comme fr-FR-DeniseNeural. Ces voix sont plus naturelles que les voix robotiques classiques. Le fichier audio généré est ensuite lu par pygame ou, si nécessaire, par un lecteur système. Un gain audio premium est appliqué afin de rendre l'alarme plus forte et plus claire.

- Débit recommandé : 155 mots par minute, pour une diction naturelle.
- Volume configuré au maximum côté moteur vocal.
- Gain audio premium réglé à 1.2 pour renforcer l'audibilité.
- Tonalité centrée à 100 pour éviter une voix trop grave ou trop aiguë.
- Court signal sonore optionnel avant la lecture de la tâche.
- Fallback automatique si la voix neurale, Internet ou un lecteur audio est indisponible.

4.4 Robustesse du déclenchement

Le scheduler ne dépend pas d'une égalité exacte entre l'heure actuelle et l'heure de la tâche. Il déclenche dès que la date et l'heure prévues sont inférieures ou égales au moment courant. Cette approche évite de rater une alarme à cause d'un décalage de quelques secondes. Une tâche n'est désactivée qu'après une lecture réussie du message principal.

5. Tests et validation

Plusieurs vérifications ont été réalisées pour valider le bon fonctionnement du projet. Les tests Django confirment la sélection d'une voix française, le comportement des réglages vocaux et la mise en file de plusieurs tâches dues au même moment. La commande `python manage.py check` ne signale aucune erreur de configuration.

- Vérification de la configuration Django : aucune erreur détectée.
- Vérification des migrations SQLite : base de données mise à jour.
- Tests unitaires : sélection de voix française et déclenchement de plusieurs alarmes.
- Installation des dépendances premium edge-tts et pygame pour améliorer la voix.
- Profil local réinitialisé sur une voix claire : Denise Neural, débit 155, volume 1.0, tonalité 100.

6. Résultats obtenus

Le projet aboutit à une application fonctionnelle capable de planifier des tâches depuis une page web et de les déclencher automatiquement par la voix. Le système est conçu pour lire autant d'alarmes qu'il y a de tâches arrivées à échéance. Les tâches simultanées sont traitées dans l'ordre, sans bloquer l'interface et sans erreur silencieuse.

La qualité vocale a été améliorée par l'ajout d'un moteur premium optionnel. Lorsque les conditions sont réunies, la voix neurale française donne un rendu plus naturel et plus professionnel. Si ce moteur n'est pas disponible, l'application conserve sa fiabilité grâce à pyttsx3 et aux moteurs vocaux du système.

7. Limites et perspectives

La principale limite du moteur neural est qu'il dépend d'une connexion Internet. De plus, la disponibilité des voix locales varie selon le système d'exploitation et les paquets installés. Pour une utilisation en production, il serait possible d'ajouter un service séparé pour le scheduler, une authentification utilisateur, des notifications visuelles ou mobiles, ainsi qu'un tableau de statistiques des alarmes déclenchées.

8. Conclusion

Ce projet démontre l'intégration pratique de plusieurs technologies autour d'un besoin concret : aider l'utilisateur à ne pas oublier ses tâches importantes. Il combine une application web Django, une base SQLite, une interface HTML/JavaScript et un système de synthèse vocale robuste. Le résultat est un planificateur simple à utiliser, mais techniquement complet, capable de surveiller les échéances et de lire les tâches à haute voix en français.

Au-delà de l'aspect fonctionnel, le projet met en évidence des notions importantes : gestion du temps, accès à la base de données, programmation concurrente avec threads, file d'attente thread-safe, fallback multiplateforme, configuration utilisateur et amélioration progressive de la qualité audio. Il constitue ainsi

une réalisation pertinente pour un stage ou un travail académique en développement logiciel.